

Introduction to Transformers

NASA AI/ML STIG Lecture Series

Helen Qu

Flatiron Institute

February 9, 2026

What problem are we solving?

- We observe **sequences**: text, spectra, light curves, images over time
- Given observations $(x_1, x_2, \dots, x_T)$, we want:

$$p(x_{t+1} \mid x_{1:t})$$

- This is **autoregressive modeling**
- Classic tools:
 - RNNs / LSTMs (sequential, hard to parallelize)
 - CNNs (local context)

Why transformers?

- Sequences often require **long-range dependencies**
 - matching brackets
 - semantic context
 - delayed causal effects
- RNNs compress history into a single hidden state
- Transformers instead learn:

which past tokens matter for predicting the next one

- Key idea: **attention as learned relevance weighting**

Language modeling as density estimation

- Given a sequence of discrete symbols:

$$x_1, \dots, x_T \in \{1, \dots, V\}$$

- Factorize the joint distribution:

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t \mid x_{1:t-1})$$

- Training objective:

$$\mathcal{L} = -\mathbb{E}[\log p_{\theta}(x_t \mid x_{1:t-1})]$$

(This is just maximum likelihood.)

Tokenization and embeddings

- Tokens are discrete $\rightarrow$ neural nets want vectors
- Learn an embedding matrix:

$$E \in \mathbb{R}^{V \times d}$$

- Each token index maps to a vector:

$$x_t \mapsto \mathbf{e}_t \in \mathbb{R}^d$$

- In the notebook:
 - Character-level vocabulary
 - Integer encoding
 - Lookup table

But embeddings lose order

- Attention is permutation-invariant
- We must explicitly encode position
- Add positional embeddings:

$$\mathbf{h}_t = \mathbf{e}_t + \mathbf{p}_t$$

- $\mathbf{p}_t$ can be:
 - sinusoidal
 - learned (used in GPT-style models)

Self-attention: core idea

Each token asks:

“Which other tokens should I look at?”

- Each token produces:
 - Query q
 - Key k
 - Value v
- All are linear projections of embeddings

Scaled dot-product attention

Given matrices:

$$Q, K, V \in \mathbb{R}^{T \times d}$$

Attention weights:

$$A = \text{softmax} \left(\frac{QK^{\top}}{\sqrt{d}} \right)$$

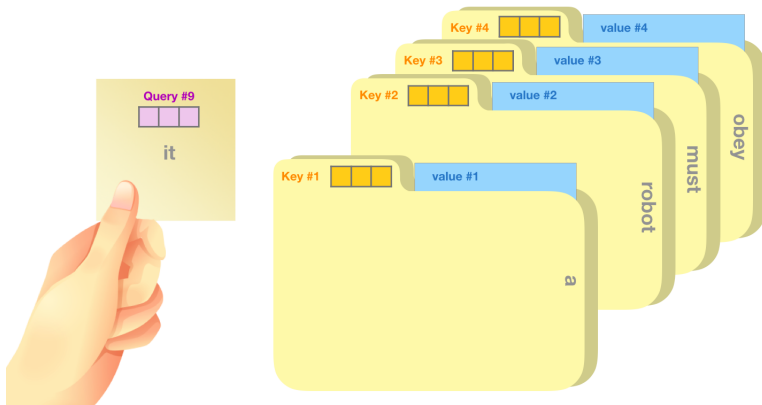
Output:

$$\text{Attention}(Q, K, V) = AV$$

Interpretation:

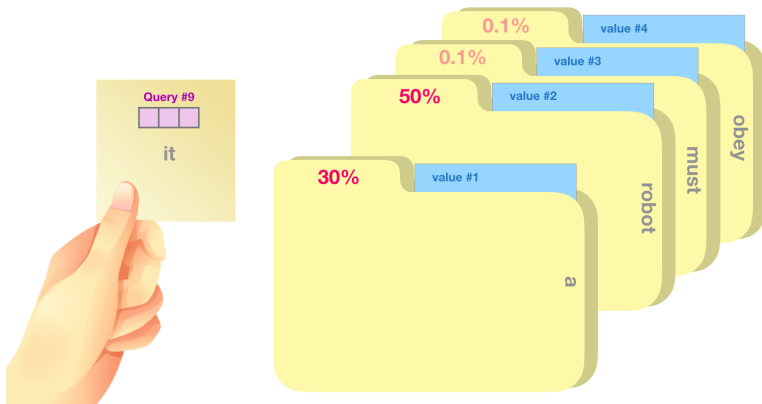
- Dot products measure similarity
- Softmax produces normalized relevance weights

Scaled dot-product attention



Source: The Illustrated GPT-2 (Jay Alammar)

Scaled dot-product attention

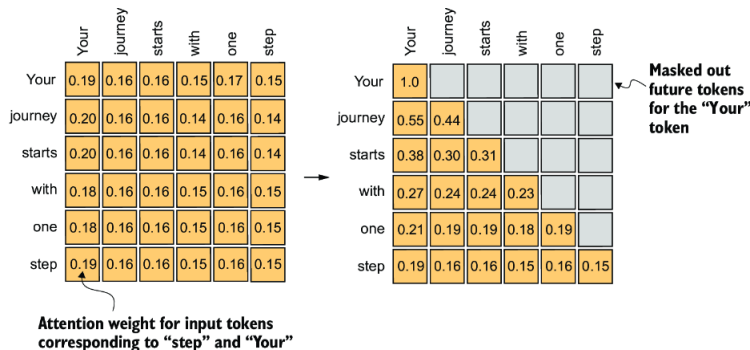


Source: The Illustrated GPT-2 (Jay Alammar)

Causal (masked) self-attention

- Autoregressive modeling requires x_t cannot see x_{t+1} :
- Apply a triangular mask:

$$QK^T \rightarrow QK^T + M, M_{ij} = -\infty \text{ for } j > i$$



Why multi-head attention?

- Single attention head = single similarity notion
- Multiple heads learn:
 - syntax
 - semantics
 - position
 - long vs short context
- Compute attention in parallel subspaces

Transformer block

A single block contains:

- ① LayerNorm
- ② Masked multi-head self-attention
- ③ Residual connection
- ④ LayerNorm
- ⑤ MLP (position-wise feedforward)
- ⑥ Residual connection

Stacking blocks = transformer

- Repeat blocks N times
- Each layer refines representations
- Final linear head projects to vocabulary:

$$\mathbb{R}^d \rightarrow \mathbb{R}^V$$

- Start with a prompt
- Model outputs logits $\ell \in \mathbb{R}^V$
- Convert to probabilities with temperature hyperparameter T :

$$p_i = \frac{\exp(\ell_i / T)}{\sum_j \exp(\ell_j / T)}$$

- (Optional) **Top-k sampling**: restrict to top- k most likely tokens
- Sample:

$$x_{t+1} \sim p(\cdot \mid x_{1:t})$$

- Append and continue

Big picture takeaways

- Transformers are flexible sequence density estimators
- Attention encodes learned relevance
- Removing the need for recurrence (in, e.g., RNNs) makes transformers massively parallelized and performant
- The transformer architecture has been extended to images, video, multimodal data

Live walkthrough: Tiny Shakespeare Transformer